HPC Project

Nicolas Hrubec - Group 24

June 10, 2024

This is my lab report for the HPC Project for the summer term 2024. Section 1 contains the specification, section 2 the description, section 3 the result discussion including the plots and section 4 the profiling output.

1 Specification

1.1 What I solved

As far as I was able to verify, the stencil computation works sequentially, parallel with point-to-point communication as well as with neighborhood collectives. There is three separate self-contained files containing the full implementations for each approach. If verification is turned on for the parallel version the sequential version is just run alongside (will be explained a bit more later), which required a lot of copy-pasting and should definitely be refactored into a shared file if I were to continue working on this, but I leave it as is for now.

1.2 Compilation

A Makefile is provided for compilation. The most important targets are:

- seq: Compiles the sequential program.
- par: Compiles the parallel program with point-to-point communication.
- par_collective: Compiles the parallel program with neighborhood collectives.

In addition, there is clean targets for individual binaries (clean_seq, clean_par, clean_par_collective) and a clean target to delete all produced binaries.

1.3 Input Parameters

Sequential

- -n, –number: Specifies the grid size. Requires an argument. Default: 10
- -g, –num_generations: Specifies the number of generations to run. Requires an argument. Default: 1000
- -s, –seed: Specifies the seed for reproducibility. Requires an argument. Default: 42
- -d, –density: Specifies the percentage of alive cells in the initial grid. Default: 28
- -v, –verbose: Enables debugging output.

Parallel Point-to-Point Communication The input parameters for this program are a superset of the input parameters for the sequential program. Additionally, the parallel point-to-point program has two flags:

- -x, –verify: Enables verification of the parallel implementation against the sequential implementation. No argument.
- -w, –weak_scaling: If this flag is turned on the grid size (as given by n) refers to the input size for each processor (not the overall grid size). No argument.

Parallel Neighborhood Collectives The input parameters for the parallel neighborhood collectives program are exactly the same as for the parallel point-to-point program and behave as described in the preceding sections.

1.4 Experiment Runs

To run the strong scaling experiments in the assignment only the input size and number of processes has to be adjusted. On hydra you can run e.g. using srun as shown below (with num nodes and num tasks per node set to 1 and input size set to 10240 in all cases for demonstration).

Of course, to run the weak scaling experiments, the -w flag needs to be set in addition.

Sequential

```
srun -t 15 -p q_student -N 1 --ntasks-per-node=1 ./seq -n 10240
```

Parallel Point-to-Point Communication

```
srun -t 15 -p q_student -N 1 --ntasks-per-node=1 ./par -n 10240
```

Parallel Neighborhood Collectives

```
srun -t 15 -p q_student -N 1 --ntasks-per-node=1 ./par_collective -n 10240
```

2 Description

2.1 Sequential

Setup and input generation The implementation of this solution is rather straightforward. First, the seed is set for reproducibility. Then two placeholder matrices are allocated for the current and next generation (same way it was done in the mpi_rand.c demonstration program such that we can use matrix[i][j] indexing). The matrix for the current generation is then filled with random initial values using the rand() function.

Generation loop overview Then for each generation the current generation is taken as the basis for the stencil updates and the results are stored in the next generation. After the stencil update is complete the next generation matrix is copied to the current generation.

Stencil Updates The main computation is happening in run_generation. For each cell the cell state is determined (dead or alive) and then the number of alive neighbors is determined. The stencil_plus/minus_operator methods implement the formulas to get the neighborhood indices as described in the assignment, given that the indices should wrap around at the edges. To avoid the modulo I use the formula:

```
x - (y * (x/y))
```

However, this is not ideal since divisions are expensive as well. I tried to replace it with a bit operation (x & (y - 1)), but this only works for powers of two, hence failing for the 10240 input size.

After the number of alive neighbors is calculated the new cell state can be decided. This could trivially be implemented using conditionals, but the implementation should be branch-free, so this is not an option. As an alternative a simple two-dimensional state lookup table is used. The first dimension describes whether the cell was dead or alive and the second dimension is used to look up the number of alive neighbors. The table contains pre-computed values for each case and so the structure can simply be indexed with the relevant values for the current index to get the updated cell state.

Optimizations Originally, one generation took about 0,2 seconds for me locally but 1,6 seconds on hydra. I then experimented with the following changes to improve the performance a bit (all runtime numbers in this section are only informal single run observations since I was looking for major performance jumps, more rigorous experimental results will be shown later in the report):

1. Precompute the indices in the stencil computation, which made almost no difference. I kept this change because the code is more readable.

2. Change the loop order in the run generation method: About 3x slower, so I removed it.

3. Use the march=native compiler flag to get architecture specific optimizations: Made no difference, so I removed it.

4. Initially I used a different matrix layout, allocating the matrix as:

```
(uint8_t *)malloc(n * n * sizeof(uint8_t));
```

I changed this to the same layout used in the mpi_rand.c as well as my parallel versions of the program. This made the verification simpler (in terms of code readability) and also improved performance on hydra to around 1,25 seconds/generation.

5. The last optimisation I tried was the above-mentioned bit operation to replace the modulo, which decreased runtime/generation to around 0,4 seconds on hydra. Unfortunately, I then noticed that this does not work for the 10240 input size and so I removed it again.

6. The last optimization I did is based on the observation that the modulo calculation is only needed for the outermost rows/cols of the matrix. Therefore, I use two stencil update methods, one with a wrap around using the modulo replacement and another with a simple index lookup. I then split the main loop (with the update computations) into several parts to segregate the matrix based on whether we need a modulo calculation or not. This gave me a significant speedup to around 0,4 seconds/generation (from 1,25 seconds/generation). The code get definitely more complex but in my opinion in this case the speedup justifies the added complexity.

Estimated Runtime Complexity / Generation Each generation runs a nested loop to access all elements in the matrix, which has size n * n. All operations in loop have a complexity of $O(1)$. So we get: $O(n^2)$.

2.2 Point-to-Point Communication

Setup In this solution first the processes are distributed in a two-dimensional grid with MPI_Dims_create and then initialize both a reordered and not reordered communicator. The communicators are then compared to see if actually any reordering took place (this is only shown if the debug flag is set). The reordered communicator is used consequently for the communication. Then the input sizes for each process are calculated. If weak scaling is turned on, the input size for each process is just the input grid size. If weak scaling is not turned on, the whole input size is split into submatrices

based on the process grid dimensions. Also the offset of the local matrix in the global matrix is calculated.

Input generation The input generation is done distributed on each process. To achieve that each process calls the fill_matrix _par method. Then each process goes through indices of the global matrix and generates a random value for each entry, but only puts the generated value in the local matrix if the value also belongs to the local matrix for the process. All values need to be generated to guarantee that the random values generated for each local submatrix are the same as if we were to generate them sequentially on one core.

Neighbor Communication The parallel updates are based on the observation that each process needs the local submatrix and the two adjacent rows/columns to do the stencil computations. This is done in two communication rounds. In the first communication round the two adjacent "top" and "bottom" rows are communicated using two MPI_Sendrecv operations. In the second round we use the updated matrix (with the previously communicated "top" and "bottom" rows added) and exchange the two adjacent columns. By using this solution we get the diagonal values "for free" without additional coding effort. For the communication, buffers with manual packing and unpacking are used instead of vector datatypes. The neighboring processes are determined using MPI_Cart_shift.

Stencil Updates Following the communication the stencil updates are done with the updated matrix (local matrix + 2 communicated rows/cols). By only going through the values that belong to the local matrix no "wrap-around" is needed and therefore also no modulo, rather the neighborhood values can be calculated directly by simply looking up the indices. Other than that the local matrix update is done similar to how it is done in the sequential solution.

Time measurement The time measurement is done exactly as described in the assignment text. The processes are synchronized before the main loop running the generation updates. After all generations are run the run time on each process is calculated and with MPI_Reduce the runtime of the slowest process is found and reported.

Verification For verification all the sequential code is put in the program as well. If the verification flag is turned on matrices for the sequential current and next generation are allocated and updated using the sequential update methods on each generation. This means in each generation the sequential update is done for the global matrix on each core. Then on each process the local process matrix is compared to the subpart of the global sequentially calculated matrix that is relevant for the process. Afterwards all verification results are accumulated using MPI_Reduce with MPI_MIN to get the global verification result. Currently the verification is only output for the input and the last generation but could easily be switched to every generation.

Decisions The most important problem I encountered while implementing this solution was the communication of the diagonal values. Originally I implemented one communication round to exchange the needed rows and cols, which obviously led to empty diagonals. I then thought about possible ways to do this and during my research I found the following stackoverflow post, describing an approach that is more elegant, readable and also requires less communication (which is why I decided to go with this approach)[1].

I did not really spend time thinking about whether I should use communication buffers or a vector datatype for the communication, instead directly opted for buffers. Changing the implementation to use vector datatypes could potentially improve the readability of the code since the required manual packing and unpacking for communication buffers would not be needed in that case. In hindsight I should have probably spend more time researching these two options.

Estimated Runtime Complexity / Generation The estimated sequential complexity is $O(n^2)$. The work is distributed evenly across the p processes: $O(n^2/p)$

This ignores communication cost.

2.3 Neighborhood Collectives

Baseline When I started to implement this approach I used the point-to-point implementation as a baseline, mainly just removing everything related

[1] https://stackoverflow.com/questions/51401874/collective-diagonal-neighborhood-communication

to the process communication. For this reason, below I will only describe everything I changed from the point-to-point approach. For anything I do not specifically mention, assume it works the same as for the point-to-point version.

Communication Setup As in the point-to-point approach communication is done in two rounds in order to get the diagonals without additional effort. To do this I define two neighborhoods, one for the first communication round (exchanging two adjacent rows) and another one for the second communication round (exchanging two adjacent columns). This is implemented by first determining the neighbors for each neighborhood with MPI_Cart_shift and then creating them using MPI_Dist_graph_create_adjacent.

Communication The actual communication is done using buffers again, therefore using alltoallv. I decided to do this because I had opted for buffers in the point-to-point approach as well, therefore I could partially reuse my implementations for packing/unpacking buffers from the previous exercise. However, I did have to adapt the code since for the neighborhood I have one buffer for all communicated elements, whereas in the point-to-point approach I had e.g. two receive buffers for the first communication round (separated into "top" and "bottom" rows received).

Estimated Runtime Complexity / Generation The estimated sequential complexity is $O(n^2)$. The work is distributed evenly across the p processes: $O(n^2/p)$

This ignores communication cost.

3 Results

In this section I provide the runtime, speedup and parallel efficiency plots for my experimental runs and discuss the results. All results are averaged over 10 runs. I also included screenshots of my raw run results in appendix A.

Sequential Average run result for N=1024: 3,53 s

Average run result for N=10240: 382,35 s

The average run result for N=10240 is by a factor of 108 slower than the run result for N=1024. Given that the instance size differs by a factor of 100 this is approximately the difference we would expect.

Strong Scaling Results The plots for the strong scaling results can be seen in Figures 1-12, where 1-3 is for N=1024 with point-to-point communication, 4-6 is for N=10240 with point-to-point-communication, 7-9 is for N=1024 collectives communication and 10-12 for N=10240 collectives communication. The x-axis refers to the number of processes involved.

N=1024: We can observe good speedups by increasing the number of cores up to 1024. However, the parallel efficiency decreases as we increase the number of processors involved. This intuitively makes sense since the instance size for each process goes down to 1 as we increase the number of cores to 1024, so the communication overhead becomes more relevant.

N=10240: For this configuration the speedups are close to linear, as can also be seen from the parallel efficiency. I am not sure why the drop for 32 cores exists, but for the other configurations it is close to 1. It seems to me that the problem instance is large enough to fully make use of up to 1024 cores without communication taking over.

Weak Scaling Results The plots for the weak scaling results can be seen in Figures 13 (point-to-point communication) and 14 (collectives communication). The x-axis refers to the number of processes involved.

Independent of the number of cores involved the runtime stays about the same at around 4,45 seconds.

Performance Comparison Point-to-Point/Collectives One thing I noticed consistently across all my experiments is that my experimental results for the point-to-point and collective implementations are very close.

While I did not have any expectations for what should be faster it seems odd that the runtime results are almost the same in all experiments. So I am not sure if I have made a mistake here or if this is a coincidence.

Figure 1: Runtime N=1024 point-to-point

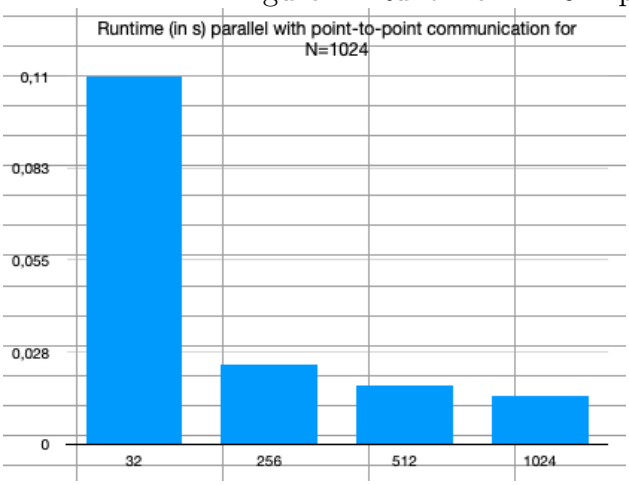


Figure 2: Speedup N=1024 point-to-point

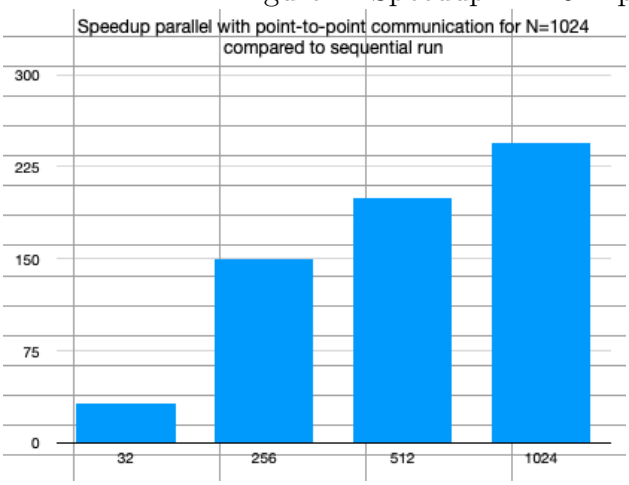


Figure 3: Efficiency N=1024 point-to-point

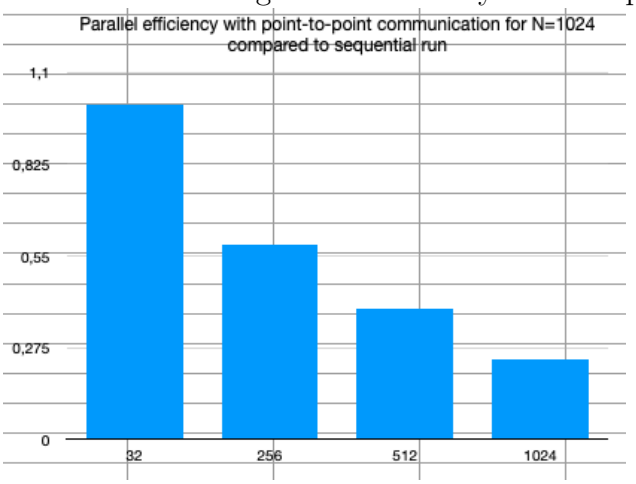


Figure 4: Runtime N=10240 point-to-point

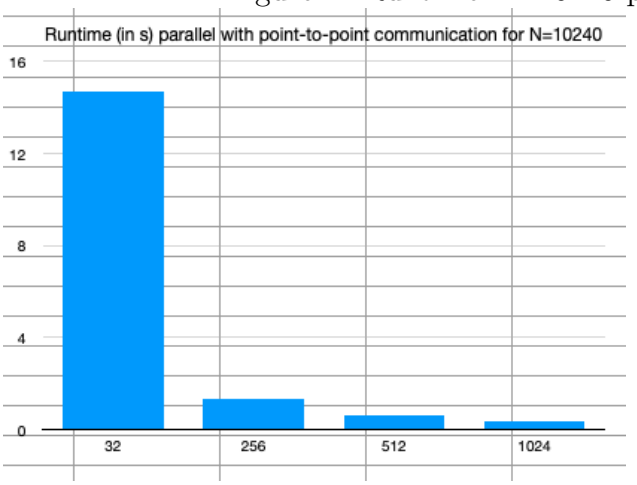


Figure 5: Speedup N=10240 point-to-point

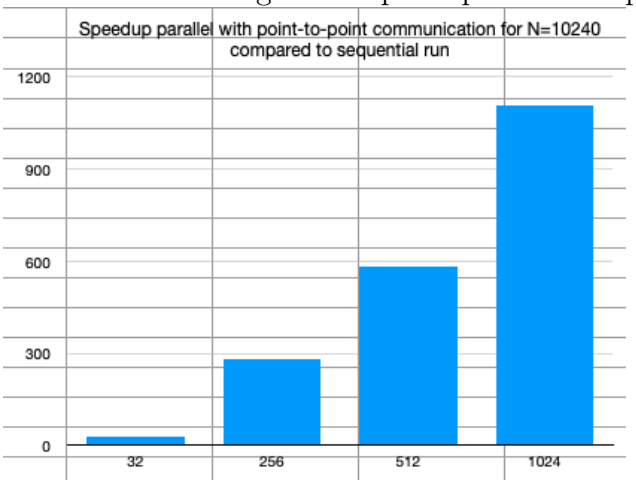


Figure 6: Efficiency N=10240 point-to-point

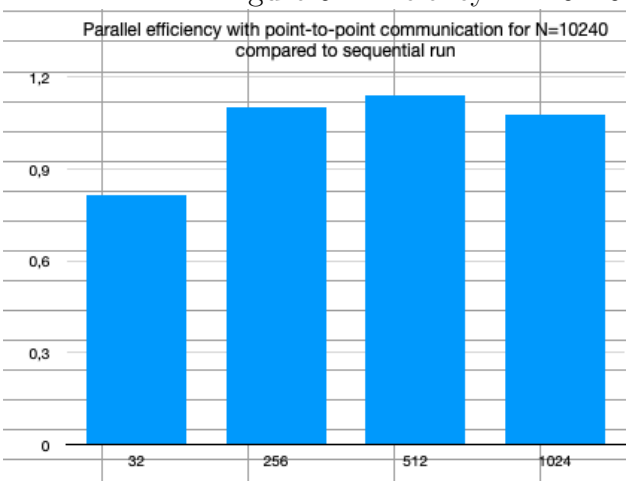


Figure 7: Runtime N=1024 collectives

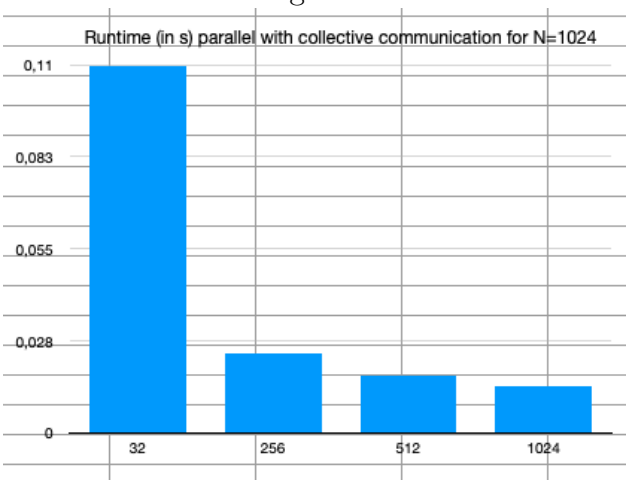


Figure 8: Speedup N=1024 collectives

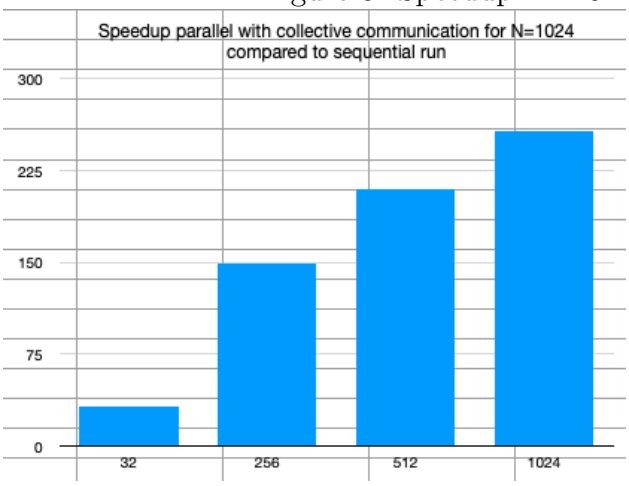


Figure 9: Efficiency N=1024 collectives

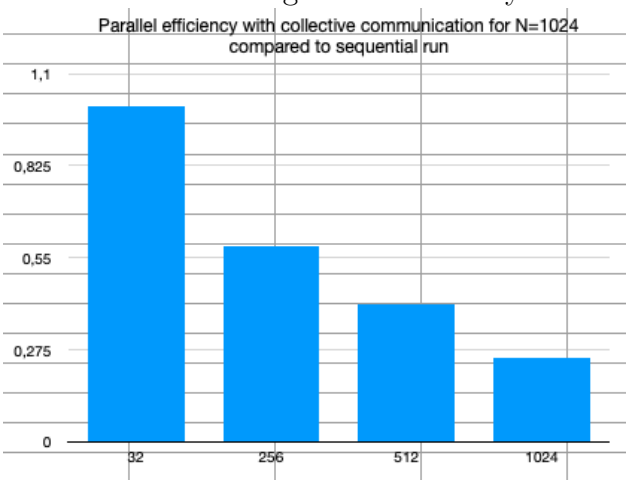


Figure 10: Runtime N=10240 collectives

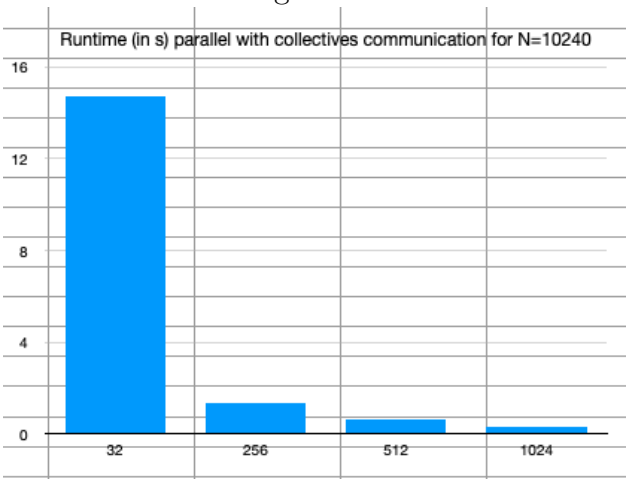


Figure 11: Speedup N=10240 collectives

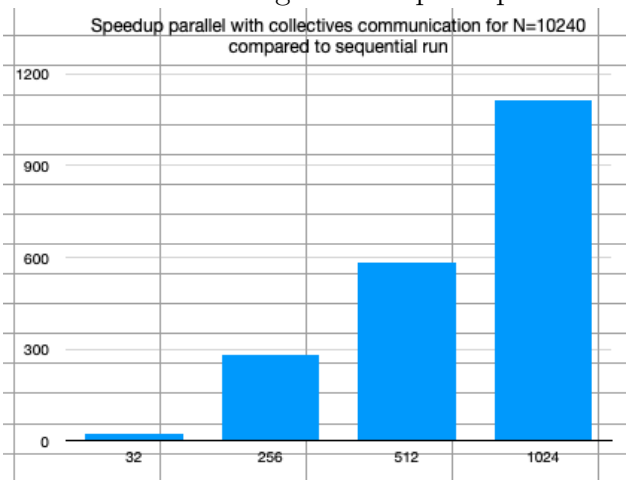


Figure 12: Efficiency N=10240 collectives

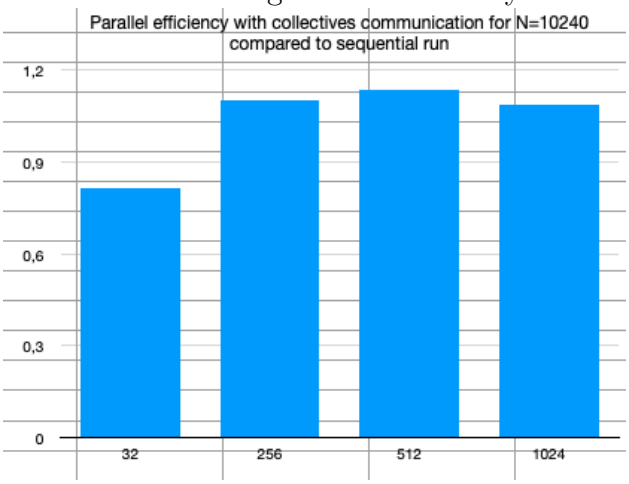


Figure 13: Runtime point-to-point N=1024 / processor

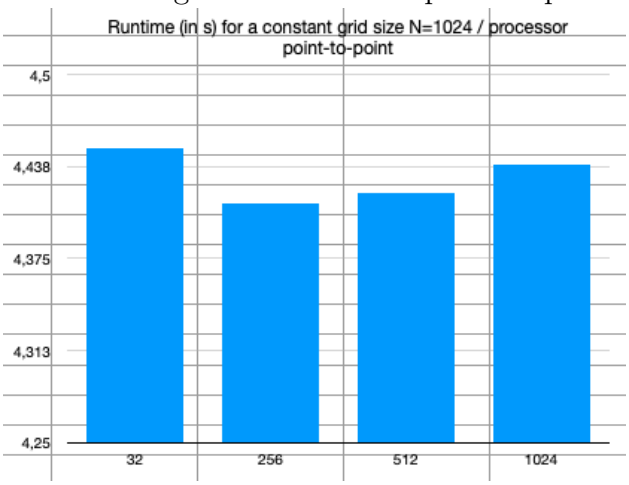
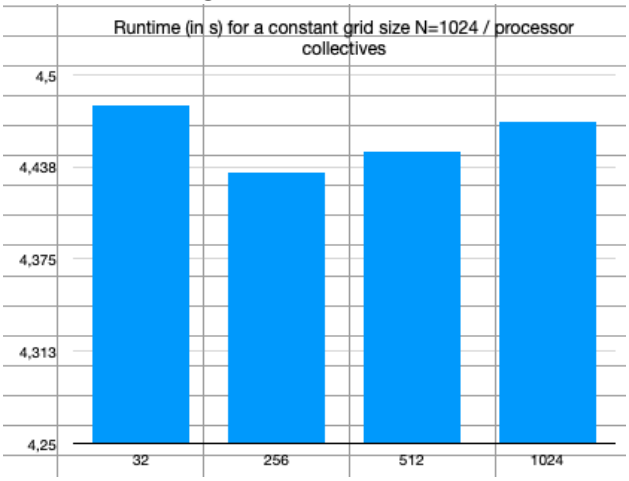


Figure 14: Runtime collectives N=1024 / processor



4 Profiling

For the profiling part I decided to run the profiler for my point-to-point approach with both input sizes for 32 and 1024 processes.

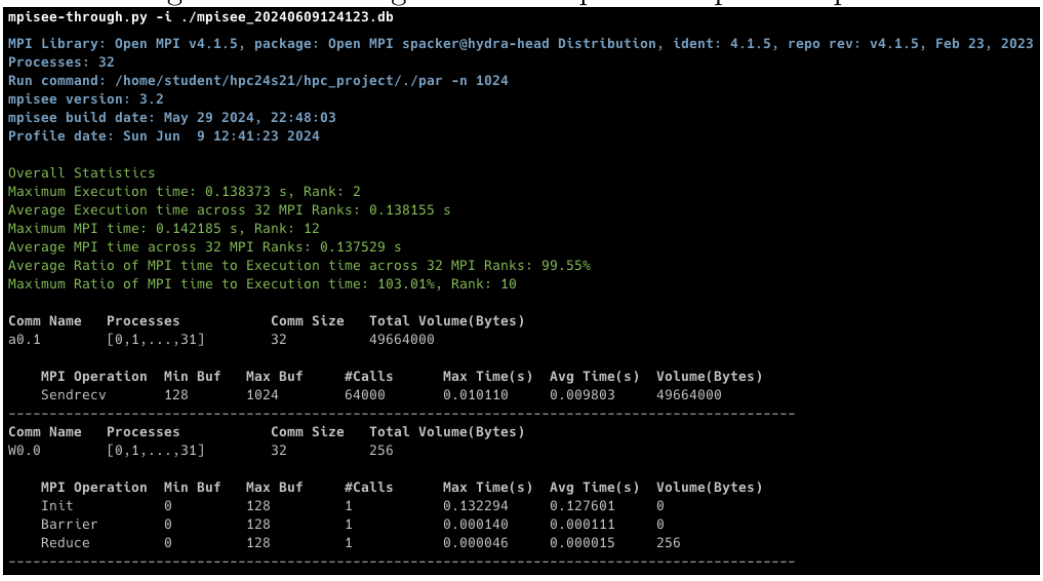
Observations For N=1024 most of the time is spent on MPI routines, partially on communication but actually mainly on MPI initialization. Interestingly, for 1024 processes the tool outputs that the MPI time is about 5 times the overall execution time. It is not immediately obvious to me how that can be. How are these values calculated exactly?

For N=10240 the MPI time makes up a much smaller share, since the program spends a lot more time on actual computation.

Overall, MPI Init and MPI Barrier take much longer if more processes are involved. In the case of N=10240 and 1024 processes the dominant routine in the MPI time (which is about 1/3 of the overall execution time) is the process synchronization before the actual computation running the gol-updates.

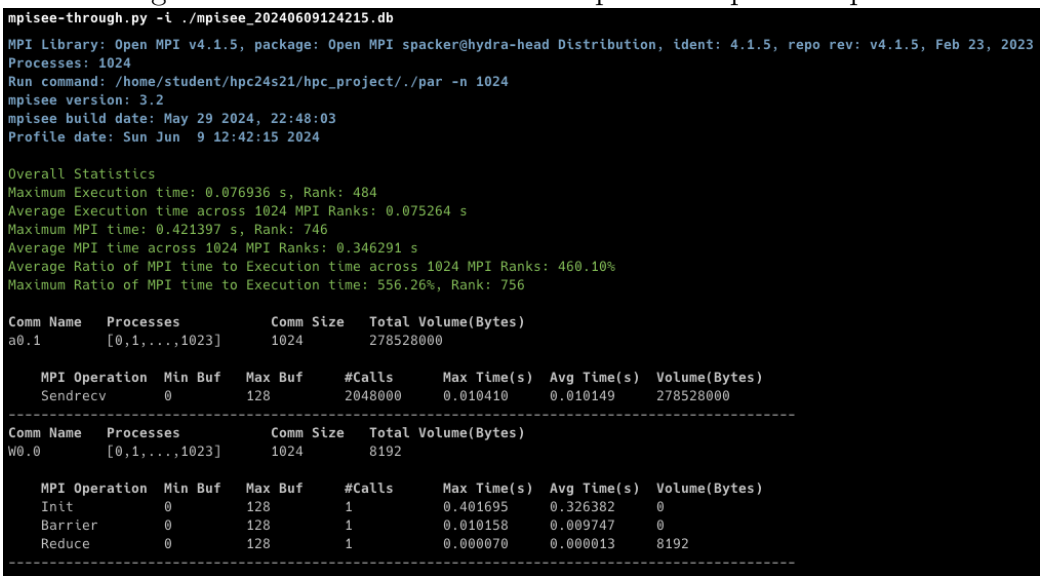
Was it useful? In this particular case I would say not really. However, I can see how it could be useful to analyze bottlenecks if I were to do further optimizations of my parallel implementations. As mentioned above, not for all values it is exactly clear to me what is measured so it would be helpful to get some more insight on that.

Figure 15: Profiling N=1024 32 processes point-to-point



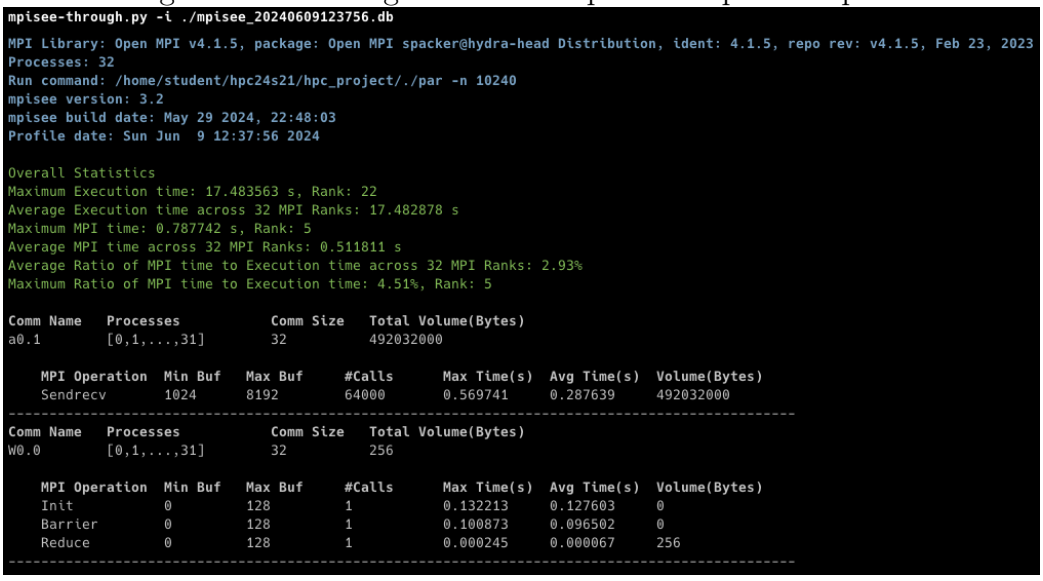
```
mpisee-through.py -i ./mpisee_20240609124123.db

MPI Library: Open MPI v4.1.5, package: Open MPI spacker@hydra-head Distribution, ident: 4.1.5, repo rev: v4.1.5, Feb 23, 2023
Processes: 32
Run command: /home/student/hpc24s21/hpc_project/./par -n 1024
mpisee version: 3.2
mpisee build date: May 29 2024, 22:48:03
Profile date: Sun Jun  9 12:41:23 2024

Overall Statistics
Maximum Execution time: 0.138373 s, Rank: 2
Average Execution time across 32 MPI Ranks: 0.138155 s
Maximum MPI time: 0.142185 s, Rank: 12
Average MPI time across 32 MPI Ranks: 0.137529 s
Average Ratio of MPI time to Execution time across 32 MPI Ranks: 99.55%
Maximum Ratio of MPI time to Execution time: 103.01%, Rank: 10

Comm Name   Processes           Comm Size   Total Volume(Bytes)
a0.1        [0,1,...,31]        32          49664000

    MPI Operation  Min Buf   Max Buf     #Calls      Max Time(s)  Avg Time(s)  Volume(Bytes)
    Sendrecv       128       1024        64000       0.010110     0.009803     49664000
-----------------------------------------------------------------------------------------
Comm Name   Processes           Comm Size   Total Volume(Bytes)
W0.0        [0,1,...,31]        32          256

    MPI Operation  Min Buf   Max Buf     #Calls      Max Time(s)  Avg Time(s)  Volume(Bytes)
    Init           0         128         1           0.132294     0.127601     0
    Barrier        0         128         1           0.000140     0.000111     0
    Reduce         0         128         1           0.000046     0.000015     256
-----------------------------------------------------------------------------------------
```

Figure 16: Runtime N=1024 1024 processes point-to-point



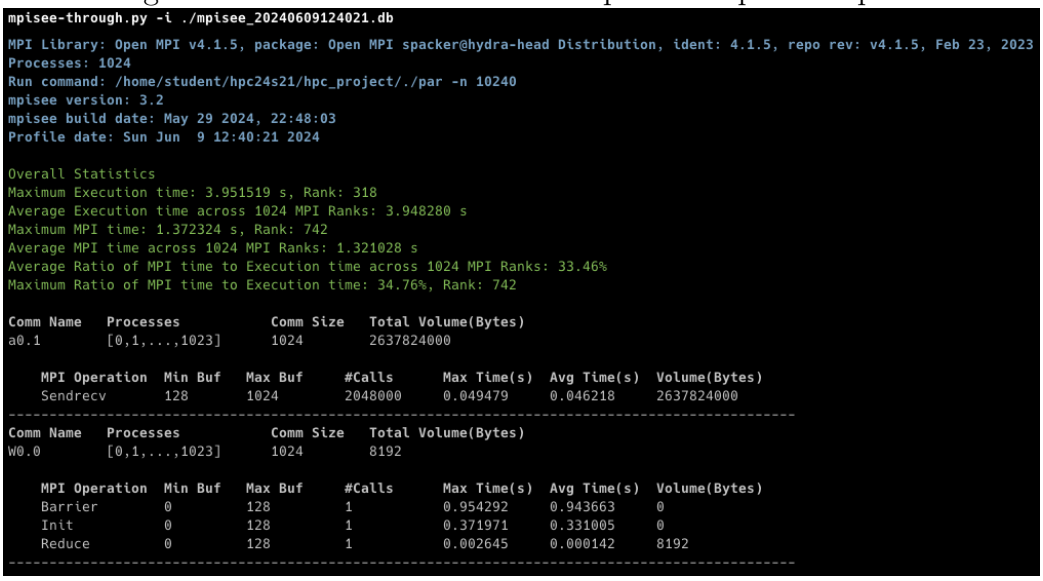
```
mpisee-through.py -i ./mpisee_20240609124215.db

MPI Library: Open MPI v4.1.5, package: Open MPI spacker@hydra-head Distribution, ident: 4.1.5, repo rev: v4.1.5, Feb 23, 2023
Processes: 1024
Run command: /home/student/hpc24s21/hpc_project/./par -n 1024
mpisee version: 3.2
mpisee build date: May 29 2024, 22:48:03
Profile date: Sun Jun  9 12:42:15 2024

Overall Statistics
Maximum Execution time: 0.076936 s, Rank: 484
Average Execution time across 1024 MPI Ranks: 0.075264 s
Maximum MPI time: 0.421397 s, Rank: 746
Average MPI time across 1024 MPI Ranks: 0.346291 s
Average Ratio of MPI time to Execution time across 1024 MPI Ranks: 460.10%
Maximum Ratio of MPI time to Execution time: 556.26%, Rank: 756

Comm Name   Processes           Comm Size   Total Volume(Bytes)
a0.1        [0,1,...,1023]      1024        278528000

    MPI Operation  Min Buf   Max Buf     #Calls      Max Time(s)  Avg Time(s)  Volume(Bytes)
    Sendrecv       0         128         2048000     0.010410     0.010149     278528000
-----------------------------------------------------------------------------------------
Comm Name   Processes           Comm Size   Total Volume(Bytes)
W0.0        [0,1,...,1023]      1024        8192

    MPI Operation  Min Buf   Max Buf     #Calls      Max Time(s)  Avg Time(s)  Volume(Bytes)
    Init           0         128         1           0.401695     0.326382     0
    Barrier        0         128         1           0.010158     0.009747     0
    Reduce         0         128         1           0.000070     0.000013     8192
-----------------------------------------------------------------------------------------
```

Figure 17: Profiling N=10240 32 processes point-to-point



```
mpisee-through.py -i ./mpisee_20240609123756.db

MPI Library: Open MPI v4.1.5, package: Open MPI spacker@hydra-head Distribution, ident: 4.1.5, repo rev: v4.1.5, Feb 23, 2023
Processes: 32
Run command: /home/student/hpc24s21/hpc_project/./par -n 10240
mpisee version: 3.2
mpisee build date: May 29 2024, 22:48:03
Profile date: Sun Jun  9 12:37:56 2024

Overall Statistics
Maximum Execution time: 17.483563 s, Rank: 22
Average Execution time across 32 MPI Ranks: 17.482878 s
Maximum MPI time: 0.787742 s, Rank: 5
Average MPI time across 32 MPI Ranks: 0.511811 s
Average Ratio of MPI time to Execution time across 32 MPI Ranks: 2.93%
Maximum Ratio of MPI time to Execution time: 4.51%, Rank: 5

Comm Name   Processes           Comm Size   Total Volume(Bytes)
a0.1        [0,1,...,31]        32          492032000

    MPI Operation  Min Buf   Max Buf     #Calls      Max Time(s)  Avg Time(s)  Volume(Bytes)
    Sendrecv       1024      8192        64000       0.569741     0.287639     492032000
-----------------------------------------------------------------------------------------------
Comm Name   Processes           Comm Size   Total Volume(Bytes)
W0.0        [0,1,...,31]        32          256

    MPI Operation  Min Buf   Max Buf     #Calls      Max Time(s)  Avg Time(s)  Volume(Bytes)
    Init           0         128         1           0.132213     0.127603     0
    Barrier        0         128         1           0.100873     0.096502     0
    Reduce         0         128         1           0.000245     0.000067     256
-----------------------------------------------------------------------------------------------
```

Figure 18: Runtime N=10240 1024 processes point-to-point



```
mpisee-through.py -i ./mpisee_20240609124021.db

MPI Library: Open MPI v4.1.5, package: Open MPI spacker@hydra-head Distribution, ident: 4.1.5, repo rev: v4.1.5, Feb 23, 2023
Processes: 1024
Run command: /home/student/hpc24s21/hpc_project/./par -n 10240
mpisee version: 3.2
mpisee build date: May 29 2024, 22:48:03
Profile date: Sun Jun  9 12:40:21 2024

Overall Statistics
Maximum Execution time: 3.951519 s, Rank: 318
Average Execution time across 1024 MPI Ranks: 3.948280 s
Maximum MPI time: 1.372324 s, Rank: 742
Average MPI time across 1024 MPI Ranks: 1.321028 s
Average Ratio of MPI time to Execution time across 1024 MPI Ranks: 33.46%
Maximum Ratio of MPI time to Execution time: 34.76%, Rank: 742

Comm Name   Processes           Comm Size   Total Volume(Bytes)
a0.1        [0,1,...,1023]      1024        2637824000

    MPI Operation  Min Buf   Max Buf     #Calls      Max Time(s)  Avg Time(s)  Volume(Bytes)
    Sendrecv       128       1024        2048000     0.049479     0.046218     2637824000
-----------------------------------------------------------------------------------------------
Comm Name   Processes           Comm Size   Total Volume(Bytes)
W0.0        [0,1,...,1023]      1024        8192

    MPI Operation  Min Buf   Max Buf     #Calls      Max Time(s)  Avg Time(s)  Volume(Bytes)
    Barrier        0         128         1           0.954292     0.943663     0
    Init           0         128         1           0.371971     0.331005     0
    Reduce         0         128         1           0.002645     0.000142     8192
-----------------------------------------------------------------------------------------------
```

A Raw Results

The below screenshots show my raw run results for all the experiments I ran. The runtimes are generally given in microseconds (except if I explicitly put a [in s] after the row description). The figures next to "Single run results" show the runtimes for individual experiments. The bottom rows are always aggregates (avg runtime, speedup, parallel efficiency) dependant on what I needed for the plots. The results should be read column-wise (so one column is one run configuration).

Figure 19: Raw results for sequential runs

	Sequential	
N	1024	10240
Processes	1	1
Single run results	3670533	382409069
	3492992	382456246
	3522131	382225437
	3508876	382474818
	3528920	382531623
	3503398	383296616
	3478703	382266246
	3617806	381983531
	3491193	381841358
	3491827	382050706
Avg run results	3530637,9	382353565
Avg run results (in s)	3,5306379	382,353565

Figure 20: Raw strong scaling results for point-to-point communication

Strong scaling	Point to Point							
N	1024	1024	1024	1024	10240	10240	10240	10240
Processes	32	256	512	1024	32	256	512	1024
Single run results	109554	23978	18132	14255	14749697	1317885	666917	392452
	109909	23508	16869	14433	14657599	1369225	659786	337206
	109947	23217	17117	17640	14717029	1362761	657035	345738
	110045	23959	22482	13992	14602709	1353310	651049	342494
	110843	23542	17280	14315	14686422	1358188	648829	338521
	109981	23188	16870	13662	14702830	1353740	654069	344080
	109642	24178	16937	13750	14644653	1356208	651501	340615
	109884	23194	16704	13613	14701008	1353974	650799	341727
	110059	23528	17273	14307	14664250	1347804	661736	338529
	109854	23479	16884	14772	14749274	1355964	654733	339716
Avg run results	109971,8	23577,1	17654,8	14473,9	14687547,1	1352905,9	655645,4	346107,8
Avg run results (in s)	0,1099718	0,0235771	0,0176548	0,0144739	14,6875471	1,3529059	0,6556454	0,3461078
Speedup	32,1049387206538	149,748607759224	199,981755669846	243,931345387214	26,0324996676947	282,616525657845	583,171276729769	1104,72391838612
Parallel Efficiency	1,00327933502043	0,584955499059469	0,390589366542668	0,238214204479701	0,813515614615459	1,10397080335096	1,13900639986283	1,07883195154895

Figure 21: Raw weak scaling results for point-to-point communication

Weak scaling	Point to Point			
N	1024	1024	1024	1024
Processes	32	256	512	1024
Single run results	4469927	4417403	4427649	4431887
	4450055	4413955	4423007	4452776
	4452192	4410492	4426899	4435692
	4443821	4416619	4416600	4439419
	4442083	4418302	4420552	4434048
	4443239	4410296	4418269	4435970
	4437426	4400915	4413152	4436182
	4457287	4412207	4416586	4433953
	4461025	4408543	4416126	4438920
	4439883	4409272	4410854	4443400
Avg run results	4449693,8	4411800,4	4418969,4	4438224,7
Avg run results (in s)	4,4496938	4,4118004	4,4189694	4,4382247

Figure 22: Raw strong scaling results for collectives communication

Strong scaling	Collectives							
N	1024	1024	1024	1024	10240	10240	10240	10240
Processes	32	256	512	1024	32	256	512	1024
Single run results	109560	23649	16688	12761	14704071	1357206	664987	341622
	109570	23592	15868	13567	14640087	1352922	663604	345175
	109729	23789	17597	13373	14742044	1354610	652068	341792
	109836	23565	17325	14321	14728225	1354742	652238	345462
	111310	23187	15989	13449	14623872	1353955	654452	339510
	109436	23798	16536	14352	14698723	1355236	655258	339225
	110527	23653	16158	15121	14618551	1352367	655970	342407
	109962	23847	16666	13420	14608190	1354211	652162	339121
	109743	24011	17923	13529	14694283	1353193	655189	345521
	109735	23521	17440	13052	14674803	1348665	653271	353678
Avg run results	109940,8	23661,2	16819	13694,5	14673284,9	1353710,7	655919,9	343351,3
Avg run results (in s)	0,1099408	0,0236612	0,016819	0,0136945	14,6732849	1,3537107	0,6559199	0,3433513
Speedup	32,113991348071	149,216349973797	209,919608775789	257,81429771076	26,0578028441334	282,448506169006	582,927221753754	1113,59288577035
Parallel Efficiency	1,00356222962722	0,582876367085144	0,409999235890213	0,251771775108164	0,814306338879169	1,10331447722268	1,1385297299878	1,0874930525101

Figure 23: Raw weak scaling results for collectives communication

Weak scaling	Collectives			
N	1024	1024	1024	1024
Processes	32	256	512	1024
Single run results	4470405	4421329	4451059	4475127
	4492662	4435212	4451223	4474590
	4490127	4441312	4446666	4468214
	4488596	4443690	4448038	4475654
	4480457	4437827	4444516	4461931
	4471099	4436071	4454937	4464924
	4471353	4430195	4444787	4461584
	4475045	4430416	4443655	4462932
	4474497	4435751	4447310	4465250
	4477641	4423398	4447279	4469740
Avg run results	4479188,2	4433520,1	4447947	4467994,6
Avg run results (in s)	4,4791882	4,4335201	4,447947	4,4679946